SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

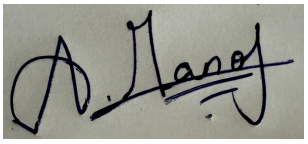
Assessment Tool Weightage: 30%

Code of Conduct:

I Manoj A / 192421295 that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question and Answers

1. A partially tested ATM withdrawal module is known only through its input-output behaviour (limits, balance validation, error messages). Reverse engineer the test design strategy, decompose the system, infer the black-box techniques used, evaluate their adequacy, and reconstruct an improved test set.

Answer:

When only inputs and outputs are visible, the test design can be reconstructed by studying which inputs were tried and which outputs were produced. The behaviour observed from the ATM suggests the following business rules: the amount must be a multiple of ₹100, the minimum withdrawal is ₹100, the maximum per transaction is ₹20,000, the daily limit is ₹40,000, the amount cannot exceed the account balance, and a specific message is shown for each failure (for example, "Invalid amount", "Limit exceeded", "Insufficient balance").

i) Decomposition into functional components

- **Card and PIN validation:** valid card, expired card, blocked card, correct PIN, wrong PIN (three attempts allowed).
- **Amount validation:** numeric input, multiple of ₹100, minimum ₹100, maximum ₹20,000 per transaction.
- **Limit validation:** cumulative daily withdrawal not above ₹40,000.
- **Balance validation:** requested amount not above available balance (and minimum balance if the bank keeps one).
- **Cash availability and dispensing:** denominations available in the machine, correct notes dispensed.
- **Account update and receipt:** balance reduced exactly once, transaction recorded, receipt printed.

ii) Equivalence classes and boundary conditions identified

- Amount classes: below ₹100 (invalid), ₹100 to ₹20,000 and multiple of 100 (valid), not a multiple of 100 (invalid), above ₹20,000 (invalid), non-numeric or blank (invalid).
- Balance classes: balance greater than the amount, equal to the amount, less than the amount.
- Daily limit classes: total after this withdrawal below, equal to, or above ₹40,000.
- Boundary values: 0, 99, 100, 200, 19,900, 20,000, 20,100 for the amount; balance = amount − 100, amount, amount + 100; daily total ₹39,900, ₹40,000, ₹40,100.

iii) Techniques inferred from the observed behaviour

The error messages for invalid, over-limit and insufficient-balance cases indicate that Equivalence Partitioning was used, because each class has its own message. Messages at exactly ₹20,000 and ₹20,100 suggest that Boundary Value Analysis was applied to the

maximum limit. There is no evidence of tests at ₹100 and ₹99, balance equal to the amount, or the daily limit, so BVA was applied only partly. The absence of combination tests (for example a valid amount, but daily limit reached and insufficient cash in the machine) shows that no decision table was used, and no state-based tests were visible for the PIN retry sequence.

iv) Evaluation of adequacy

- Strong point: each failure message has at least one test, so the main error classes are covered.
- Gap 1: the lower boundary and the multiple-of-100 rule are untested.
- Gap 2: the daily cumulative limit and the balance-equals-amount boundary are untested.
- Gap 3: no combinations of conditions and no sequence tests (three wrong PINs, then card blocked).
- Gap 4: no negative and robustness tests such as cancel during withdrawal, power or network failure after debit, or cash jam.

v) Reconstructed improved test cases

- TC1: Amount ₹100, balance ₹5,000 → accepted (lower boundary).
- TC2: Amount ₹99 → "Invalid amount" (below minimum).
- TC3: Amount ₹150 → "Invalid amount" (not a multiple of ₹100).
- TC4: Amount ₹20,000, balance ₹50,000 → accepted (upper boundary).
- TC5: Amount ₹20,100 → "Limit exceeded".
- TC6: Amount equal to balance (₹3,000, balance ₹3,000) → accepted, balance becomes ₹0.
- TC7: Amount ₹3,100, balance ₹3,000 → "Insufficient balance".
- TC8: Third withdrawal of ₹20,000 in one day (total ₹60,000) → "Daily limit exceeded"; also tests at ₹39,900, ₹40,000 and ₹40,100 cumulative.
- TC9: Blank, text or negative amount → input rejected with a clear message.
- TC10: Three wrong PINs → card retained or blocked; correct PIN afterwards is refused.
- TC11: Valid amount but machine has insufficient notes → message, no debit.
- TC12: Power or network failure after cash is counted → transaction is rolled back or reversed and balance is consistent.
- TC13: Double press of the confirm button → only one debit.

vi) Conclusion

The reconstruction shows that the original design relied on equivalence classes with limited boundary testing. The improved set adds the missing boundaries, cumulative-limit checks,

combinations and failure scenarios. It therefore covers all critical scenarios of the withdrawal function and can be documented as a traceable black-box test design.

2. A module's execution traces and code snippets are given, but there is no test documentation. Reverse engineer the white-box approach by constructing the control flow graph, identifying decisions, loops and independent paths, evaluating coverage, and reconstructing a complete path testing strategy.

Answer:

In the absence of documents, the white-box approach is reconstructed from the code and the traces. The module used in this analysis calculates an order total. Its logic, with line numbers, is as follows.

- 1: total = 0, i = 0
- 2: while (i < n) (decision D1, loop)
- 3: if (stock[i] ≤ 0) (decision D2): count a back-order, else add price × quantity to total
- 4: if (qty[i] > 10) (decision D3): apply a 5% bulk discount to this item
- 5: i = i + 1
- 6: if (total > 1000 AND member) (D4 and D5): apply loyalty discount of 10%
- 7: return total

i) Control flow graph

The graph has these nodes: Start (1), loop test (2), stock test (3), back-order (3a), add price (3b), bulk test (4), bulk discount (4a), increment (5), total test (6a), member test (6b), loyalty discount (6c) and return (7). Edges: Start → loop test; loop test → stock test (true) or → total test (false); stock test → back-order (true) or add price (false); both lead to the bulk test; bulk test → bulk discount (true) or increment (false); bulk discount → increment; increment → loop test (the back edge); total test → member test (true) or return (false); member test → loyalty discount (true) or return (false); loyalty discount → return.

ii) Decision points, loops and independent paths

The decision points are D1 (loop), D2 (stock), D3 (quantity), D4 (total above 1,000) and D5 (member). With the compound condition split into two predicates, the cyclomatic complexity is V(G) = 5 + 1 = 6, so six independent paths exist in the basis set.

- P1: loop not entered (n = 0), total test false → return 0.
- P2: loop entered, item out of stock (back-order), total ≤ 1000.
- P3: loop entered, in stock, quantity ≤ 10, total ≤ 1000.
- P4: loop entered, in stock, quantity > 10 (bulk discount), total ≤ 1000.
- P5: in stock, total > 1000, customer not a member → no loyalty discount.
- P6: in stock, total > 1000, customer is a member → loyalty discount applied.

iii) Evaluation of the existing test adequacy

The traces show only two executions: an in-stock item with a small quantity (P3) and an out-of-stock item (P2). The statements executed are initialisation, loop test, back-order, add price, increment, total test and return, which is 7 of 9 statements, or about 78% statement coverage. Of the 10 branch outcomes (two for each of the five decisions) only 6 were taken, giving 60% branch coverage, and only 2 of 6 independent paths (33%) were executed. The bulk-discount statement and the loyalty-discount statement were never executed, so any calculation error in them is undetected.

iv) Reconstructed path testing strategy

- Create one test for each of the six basis paths: n = 0; one out-of-stock item; one item with quantity 5; one item with quantity 11; total ₹1,500 for a non-member; total ₹1,500 for a member.
- Add loop-boundary tests: zero, one, two, typical and maximum items, and maximum + 1.
- Add boundary tests for the decisions: quantity 10, 11; total 1,000 and 1,001.
- Add combined-path tests, such as an order with several items where some are out of stock, some have bulk quantities and the member discount applies at the end.
- Check that the expected totals are calculated independently, and measure statement, branch and path coverage with a coverage tool until 100% statement, 100% branch and all six basis paths are achieved.

v) Conclusion

The reconstructed approach shows that the earlier testing was incomplete. The path testing strategy gives structured, measurable coverage, so that every decision outcome and every discount calculation is verified before release.

3. A web application's login and transaction system was tested with unknown techniques; only defect logs and test results exist. Reverse engineer the approach from the defect patterns, decompose the system, evaluate coverage gaps, and reconstruct a strategy integrating ECP, BVA and path testing.

Answer:

Defect logs reveal which techniques were effective, because defects are usually found where the tests were designed to look. Equally, the defects found in production and the areas with no defects recorded point to untested parts.

i) Reverse engineering from defect patterns

- Defects such as “password of exactly 8 characters rejected” and “account locked after 4 attempts instead of 3” indicate that Boundary Value Analysis was used on the password length and the attempt counter.

- Defects such as “blank username accepted” or “special characters crash the page” indicate Equivalence Partitioning with some invalid classes.
- Defects such as “transfer of ₹50,000 fails” show that limits were tested, but only at the exact maximum value.
- Defects of the type “valid login followed by session timeout displays error page” suggest that exploratory or error-guessing testing was used rather than any systematic technique.
- There are no defects about statement-level or branch-level faults (for example the wrong operator in the interest calculation), which suggests that white-box coverage was not measured.
- Duplicate payments after the browser back button show that sequence and state testing was missing.

ii) Decomposition into testable components

- Login: username and password validation, lockout, password reset, remember-me.
- Session management: creation, timeout, logout, concurrent sessions.
- Transaction: payee selection, amount validation, limit checks, OTP, confirmation.
- Back-end: API, database update, transaction log, notification.
- Non-functional: security, performance, browser compatibility.

iii) Gaps in coverage

- Boundary testing for amount, daily limit and OTP expiry is only partial (some boundaries only).
- No white-box testing: decision and loop paths in the transaction code were never measured.
- Combinations such as valid user with locked account, or valid amount with insufficient balance and expired session, were not tested.
- Negative and security tests (injection, brute force, session hijack) were missing.
- No regression suite is evident because earlier defects reappear in the logs.

iv) Reconstructed comprehensive strategy

- **ECP:** classes for username (valid, blank, too long, special characters), password (valid, short, no digit, common), amount (valid, zero, negative, above limit, non-numeric) and account state (active, locked, closed).
- **BVA:** password length 7, 8, 20, 21; attempts 2, 3, 4; amount 0, 1, 50,000, 50,001; OTP at 29, 30 and 31 seconds; session at timeout minus 1 second and plus 1 second.
- **Path testing:** draw the CFG for login and transaction functions, compute cyclomatic complexity, and test each independent path (valid login, wrong password, locked account, expired session, insufficient balance, limit exceeded, successful transfer).

- **Decision table and state tests:** combinations of account state, balance, limit and OTP result; session and transaction state transitions.
- **Process:** link tests to requirements in a traceability matrix, automate regression, add each defect from the log as a permanent test, and measure statement and branch coverage.

v) Justification

ECP and BVA remove defects at input level, path testing finds logic errors in the code, and state and decision-based tests address the combination and sequence defects seen in the logs. By addressing each pattern found in the defect history, the reconstructed approach covers the gaps and reduces the chance that the same defects return.

4. A legacy inventory management system provides only successful and failed transaction logs for stock updates and low-stock alerts. Reverse engineer the business logic to find hidden boundaries and equivalence partitions, decompose the update process, and reconstruct a black-box test design.

Answer:

Logs show the input of each stock update and the result, so the rules can be inferred by looking for the point where the result changes. Assume the logs show the following: a stock issue of 5 units from a stock of 20 succeeds; an issue of 25 from 20 fails with "Insufficient stock"; an update that leaves stock at 11 produces no alert, while an update that leaves stock at 10 and at 9 produces a low-stock alert; a receipt that would raise the stock above 1,000 fails with "Capacity exceeded"; and an update with quantity 0 or negative fails.

i) Inferred business rules and hidden boundaries

- Quantity must be a whole number of at least 1 (0, negative and decimals fail).
- A stock issue cannot be larger than the current stock, so stock never becomes negative (boundary: issue equals stock is allowed and leaves 0; issue equals stock + 1 fails).
- The low-stock alert threshold is 10: an alert is generated when the new stock is less than or equal to 10 (boundary at 10 and 11).
- The maximum capacity is 1,000 units, so a receipt is accepted when the new stock is at most 1,000 (boundary at 1,000 and 1,001).
- An alert should be generated once when stock crosses the threshold, not repeated with every later update (suggested by repeated alerts in the failed log entries).

ii) Equivalence partitions

- Transaction type: stock receipt, stock issue, adjustment.
- Quantity: invalid (≤ 0, decimal, non-numeric), valid.
- Resulting stock after update: negative (invalid), 0 to 10 (alert zone), 11 to 1,000 (normal zone), above 1,000 (invalid).

- Item: existing and active, existing but discontinued, non-existing.

iii) Decomposition of the update process

- Input validation: item code, quantity, transaction type.
- Stock calculation: new stock = old stock ± quantity.
- Rule check: no negative stock, no over-capacity.
- Threshold evaluation: compare new stock with the low-stock level.
- Alert generation: create, suppress duplicates, notify purchasing.
- Persistence and audit log: save the new stock and write the log entry in a single transaction.

iv) Reconstructed black-box test design

- Quantity boundary tests: 0, 1, 2 for the quantity.
- Issue tests with stock 20: issue 19 (leaves 1, alert), 20 (leaves 0, alert), 21 (fails).
- Alert threshold tests: update leaving stock at 12, 11 (no alert), 10 (alert), 9 (alert).
- Capacity tests: stock 990, receipt 10 (stock 1,000, accepted), receipt 11 (stock 1,001, rejected).
- Invalid item and invalid type tests: non-existing item, discontinued item, blank code.
- Sequence tests: two issues in a row crossing the threshold produce only one alert; receipt that brings stock back above 10 clears the alert state; an alert is raised again when stock falls below the threshold again.
- Concurrency test: two simultaneous issues of 15 from stock 20 should result in one success and one failure, never negative stock.

v) Justification

Each test is selected from an inferred rule or a boundary where the logs show the outcome changing, so no hidden rule is left unverified. Boundary tests expose off-by-one errors in the alert and capacity checks, sequence tests expose duplicate or missing alerts, and the concurrency test checks the integrity of stock. The design therefore closes the gaps found in the logs with a minimum number of test cases.

5. A compiled library management module and its CPU/memory profile during a stress test are given. Reverse engineer the white-box testing requirements by mapping execution spikes to loops and decision clusters, construct a conceptual CFG, and reconstruct a path testing strategy for the resource-intensive paths.

Answer:

When source code is not available, the execution profile can be used to infer which parts of the logic consume most resources. The analysis below is conceptual and uses only the observed profile and the known functions of a library module.

i) Mapping spikes to probable logic

- **CPU spike during catalogue search:** typical of a loop over all books (and a nested comparison of keywords), which suggests a linear or nested loop without indexing.
- **CPU spike at overdue-fine processing:** a loop over all active loans, each with a date calculation and a decision on the fine slab (1 to 7 days, 8 to 30 days, beyond 30 days).
- **Memory growth during report generation:** a loop that loads all member records into a list and does not release them, suggesting a possible memory leak or unbounded collection.
- **Short repeated spikes at check-out:** decision cluster for member status, borrowing limit, book availability and reservation queue.
- **Memory that does not fall after the test ends:** objects kept in a cache or a resource not closed (database connection, file handle).

ii) Conceptual control flow graph

Node A: Start and read request type. Decision D1: request is search, check-out, return or report. Search branch: node S1 initialise, loop D2 (for each book in catalogue), decision D3 (title or author matches keyword), node S2 add to the result list, back edge to D2, and exit to S3 return the result. Return branch: node R1 find loan, decision D4 (returned after due date), decision D5 (fine slab), node R2 calculate fine, node R3 update record. Check-out branch: decisions D6 (member active), D7 (borrowing limit not reached), D8 (copy available), D9 (reserved for another member), node C1 issue book. Report branch: loop D10 (for each member), inner loop D11 (for each loan of the member), node G1 accumulate, and exit to G2 print report.

The number of predicate nodes is 11, so V(G) = 11 + 1 = 12. The module is therefore complex, and the largest risk lies in the three loop structures D2, D10 and D11.

iii) Path testing strategy for resource-intensive paths

- Test each of the 12 independent paths functionally, to confirm that the logic is correct.
- Loop tests for search: empty catalogue, one book, many books (for example 10,000 and 100,000), and keyword matching no book, one book and all books.
- Loop tests for reports: zero members, one member with zero loans, one member with many loans, many members each with many loans (nested loop), where the inner loop is tested first.
- Fine calculation tests at the slab boundaries: 0, 1, 7, 8, 30 and 31 days late, and bulk processing of all loans at once.
- Resource measurement during every path: record CPU, memory, query time and number of loop iterations, and compare with agreed limits (for example search under 2 seconds, memory returns to baseline after the report).

- Soak test: repeat the report and search paths for several hours to detect memory leaks, and verify that connections and files are closed after each request.

iv) Impact on system stability

Targeting the heaviest paths shows how run time and memory grow as data and users grow. Typical corrective actions that follow from these tests are indexing or caching for the search, paging for reports, and releasing resources after use. With these tests in the regression suite, the module can be shown to remain stable under stress, and the risk of slow response or crash during peak use is reduced.

6. A mobile payment gateway's JSON API responses for Pending, Success, Reversal and Timeout are given. Reverse engineer the state machine from the transition triggers and data constraints, identify the state-transition technique, and reconstruct a complete state-based test suite with a state-transition diagram.

Answer:

The JSON responses contain fields such as transactionId, status, amount, timestamp and, for reversals, a reversalReference. By comparing consecutive responses for the same transactionId, the allowed changes of status can be deduced.

i) Inferred states and triggers

- Initiated: request received (internal state, before the first response).
- Pending: waiting for the bank or user confirmation. Triggered by a valid payment request.
- Success: the bank confirms the debit. Trigger: confirmation received within the time limit.
- Timeout: no confirmation within the limit (for example 60 seconds). Trigger: timer expiry.
- Reversal: money returned to the customer. Triggers: payment cancelled after success, or a late debit after a timeout, or a duplicate charge.
- Failed: bank declines (insufficient funds or invalid account); a terminal state.

ii) Inferred data constraints

- The amount in a reversal must not be greater than the original amount, and a partial reversal leaves the remaining amount.
- A reversal needs the reversalReference and must contain the original transactionId.
- The same transactionId must never produce two successful debits (idempotency).
- Timestamps must increase with each state change.

iii) State-transition diagram (text representation)

- [Initiated] --valid request--> [Pending]

- [Initiated] --invalid request--> [Failed]
- [Pending] --bank confirms--> [Success]
- [Pending] --bank declines--> [Failed]
- [Pending] --no response within limit--> [Timeout]
- [Timeout] --late confirmation--> [Success] (or [Reversal] if the merchant has already cancelled)
- [Timeout] --status check shows no debit--> [Failed]
- [Success] --refund or cancellation--> [Reversal]
- [Reversal], [Failed] and fully refunded [Success] are final states.

iv) Technique implied

The behaviour matches State Transition Testing, in which each response is a state, each trigger an event and the data constraints are guard conditions. The set of responses given covers only the valid transitions (all-states and probably 0-switch coverage); illegal transitions and sequences are not represented.

v) Reconstructed state-based test suite

- Valid transitions: Initiated → Pending, Pending → Success, Pending → Failed, Pending → Timeout, Timeout → Success, Timeout → Failed, Success → Reversal (one test for each).
- Illegal transitions that must be rejected with an error code and no change of state: Success → Pending, Success → Timeout, Failed → Success, Reversal → Success, Reversal → Pending, Timeout → Pending.
- Two-step sequences (1-switch): Pending → Timeout → Success, Pending → Success → Reversal, Timeout → Success → Reversal.
- Guard tests: reversal of the full amount, of the amount minus 1, of the amount plus 1 (rejected), second reversal of an already reversed payment (rejected), reversal without reference (rejected).
- Idempotency test: send the same confirmation twice, expect only one Success and one debit.
- Timing tests: confirmation at 59, 60 and 61 seconds, with the expected state at each point.
- Failure tests: network loss after debit and before the response, then recovery check must give the correct final state without double charge.

vi) Conclusion

The reconstructed state machine and suite cover all states, all legal transitions, the important illegal transitions and the data guards, which gives a complete and documented model of the payment gateway and reduces the risk of lost or duplicated payments.

7. An encrypted communication module shows only black-box behaviour through input parameters and encrypted outputs. Perform fuzzing analysis to find error-handling boundaries, decompose the module, and reconstruct test cases using BVA that prevent buffer overflow and injection vulnerabilities.

Answer:

This analysis is intended for testing a module that the tester is authorised to test. Fuzzing means sending systematically varied and unexpected inputs and observing how the module responds, in order to discover where validation starts and stops.

i) Fuzzing analysis of the input fields

- **Message length:** empty, 1 byte, 1,023, 1,024 and 1,025 bytes. If errors appear first at 1,025 bytes, the maximum accepted length is inferred as 1,024 bytes.
- **Key length:** lengths of 127, 128, 129, 191, 192, 255, 256 and 257 bits. Acceptance only at 128, 192 and 256 shows that the module accepts standard key sizes only.
- **Initialisation vector (IV):** 15, 16 and 17 bytes. Acceptance only at 16 bytes indicates a fixed-size check.
- **Recipient or session identifier:** very long values, empty values, special characters, control characters, Unicode and a null byte, to see whether they are rejected or passed on.
- **Encoding and format:** invalid character encodings, truncated or malformed headers, wrong data types.
- **Observation points:** error code, error message text, response time, crashes, and whether the same input always gives a different ciphertext (random IV) or the same ciphertext (a weakness).

ii) Decomposition of the module

- Data validation component: checks length, type, format and allowed characters before processing.
- Encryption component: key handling, IV generation, encryption and authentication tag.
- Output component: encoding of ciphertext, packaging with header and length field.
- Error-handling component: error codes, logging without revealing secrets.

iii) Reconstructed test cases with BVA

- Message length: 0, 1, 2, 1,023, 1,024, 1,025 bytes (expected: 0 rejected or handled by the rule, 1 to 1,024 accepted, 1,025 rejected with a clean error).
- Key length: 127, 128, 129, 255, 256, 257 bits; unsupported lengths must be rejected.
- IV length: 15, 16, 17 bytes; reuse of the same IV with the same key must be prevented or detected.
- Identifier length at its maximum and maximum + 1, with the special characters, null byte and quotation marks inside the valid length.

- Decryption round trip: for every accepted length, decrypting the output must return the original message exactly (including boundary lengths and padding).
- Tamper tests: change one byte of the ciphertext or tag, so decryption must fail without returning partial data.
- Robustness: random and mutated inputs in large numbers, with monitoring for crashes, hangs and memory growth.

iv) How the design prevents buffer overflow and injection

- BVA at the length limit proves that input longer than the buffer is rejected before it is copied, so memory outside the buffer is never overwritten.
- Tests with special characters and command or query fragments in every text field prove that input is treated only as data, which supports validation by whitelist and the use of parameterised queries and safe APIs.
- Error tests show that messages do not reveal internal details, keys or memory content.
- Monitoring with memory-checking tools (AddressSanitizer, Valgrind) during the fuzz run detects overflows that do not cause visible failures.

The reconstructed design therefore documents the exact validation limits, tests each of them at and around the boundary, and gives evidence that malformed input cannot corrupt memory or alter the behaviour of the module.

8. Legacy unit tests for an insurance premium calculator show 100% statement coverage but miss defects in multi-condition if statements. Reverse engineer the original white-box approach, identify the inadequacy, construct a new CFG and independent paths, and reconstruct a strategy achieving 100% branch and path coverage.

Answer:

A report of 100% statement coverage means that every line was executed at least once, but it does not show that every outcome of every decision, or every combination of conditions, was tested. This is the likely reason why defects in compound conditions escaped.

i) Module analysed

- 1: premium = basePremium
- 2: if (age < 25 OR claims > 2): premium = premium × 1.5 (decision D1, conditions A and B)
- 3: if (vehicleAge > 10 AND coverage = comprehensive): premium = premium × 0.9 (decision D2, conditions C and D)
- 4: if (premium > 20,000): premium = 20,000 (decision D3, cap)
- 5: return premium

ii) Original approach inferred and its inadequacy

One test with a young driver, an old vehicle with comprehensive cover and a high base premium makes all three conditions true and executes every statement (lines 1 to 5). The original team therefore used statement coverage as the only criterion. This is inadequate because the false outcomes of D1, D2 and D3 are never taken, no individual condition is shown to affect the result (for example if OR was coded instead of AND, or claims > 2 coded as claims >= 2), and the order of the adjustments, which matters for the cap, is not tested.

iii) Control flow graph and independent paths

The graph consists of nodes for the start, D1, the loading statement, D2, the discount statement, D3, the cap statement and the return. Each decision has a true edge through its statement and a false edge that skips it, and both join before the next decision. At decision level, V(G) = 3 + 1 = 4 basis paths. If every condition is counted as a predicate (A, B, C, D and the cap) then V(G) = 5 + 1 = 6. Because the decisions are in sequence, the total number of distinct paths is 2 × 2 × 2 = 8, and all of them can be tested.

iv) Reconstructed strategy for 100% branch and path coverage (cap ₹20,000)

- T1: D1 false, D2 false, D3 false: age 40, claims 0, vehicle 5 years, third-party, base ₹10,000 → expected ₹10,000.
- T2: D1 false, D2 false, D3 true: age 40, vehicle 5 years, base ₹25,000 → expected ₹20,000 (capped).
- T3: D1 true, D2 false, D3 false: age 22, vehicle 5 years, base ₹10,000 → expected ₹15,000.
- T4: D1 true, D2 false, D3 true: age 22, vehicle 5 years, base ₹15,000 → ₹22,500 capped to ₹20,000.
- T5: D1 false, D2 true, D3 false: age 40, vehicle 12 years, comprehensive, base ₹10,000 → ₹9,000.
- T6: D1 false, D2 true, D3 true: age 40, vehicle 12 years, comprehensive, base ₹25,000 → ₹22,500 capped to ₹20,000.
- T7: D1 true, D2 true, D3 false: age 22, vehicle 12 years, comprehensive, base ₹10,000 → ₹13,500.
- T8: D1 true, D2 true, D3 true: age 22, vehicle 12 years, comprehensive, base ₹15,000 → ₹20,250 capped to ₹20,000.

These eight tests give 100% statement, 100% branch and 100% path coverage. Two tests (all conditions true and all false) would already give 100% branch coverage, but they would not show the effect of each condition, so the following condition tests are added (MC/DC style) for D1 and D2: for D1, (age 22, claims 0), (age 40, claims 3) and (age 40, claims 0); for D2, (vehicle 12, comprehensive), (vehicle 12, third-party), (vehicle 10, comprehensive) and (vehicle 11, comprehensive). Boundary tests are also included at age 24, 25, claims 2, 3, vehicle age 10, 11 and premium 19,999, 20,000, 20,001.

v) How this reduces the risk of calculation errors

- Each outcome of each decision is executed with an independently calculated expected premium, so wrong operators and constants are exposed.
- Condition tests show that each factor really influences the premium, which detects AND/OR mix-ups and wrong comparison operators.
- Order and cap behaviour are tested in combination, which prevents errors in the interaction of loadings, discounts and the cap.
- Because a wrong premium has direct financial and legal effect, the use of branch, condition and path coverage gives measurable assurance that the calculator is correct, and the tests become a regression suite for future tariff changes.

9. A cloud file synchronisation tool provides activity logs and a list of "lost file" defects. Reverse engineer the synchronisation logic from defect patterns related to file size, type and network speed, infer the ECP used, and reconstruct a strategy integrating BVA with stress testing.

Answer:

The defect reports are the main source of information. By grouping the lost-file cases by size, type and network speed, the points where the algorithm behaves differently can be estimated.

i) Inferred algorithm logic from the defect patterns

- **File size:** most lost files are larger than about 2 GB, and some between 100 MB and 150 MB. This suggests that small files are uploaded in one request, files above 100 MB are split into chunks, and a 32-bit size field or a chunk counter fails at 2 GB.
- **File type:** files with temporary or lock extensions (such as ~$ or .tmp), zero-byte files, files with long names or special characters, and files open in another application are lost. This suggests a filter list that skips some files and an incorrect handling of locked or special-name files.
- **Network speed:** losses mostly occur on slow or unstable connections (below 1 Mbps or with packet loss). This suggests a fixed timeout, and failed chunks that are marked as completed or not retried.
- **Conflicts:** the same file edited on two devices is overwritten; this suggests that the last-writer-wins rule is used without keeping versions.

ii) Equivalence classes probably used by the developers

- Size: small (under 100 MB), large (100 MB to 2 GB), and, probably missing, very large (above 2 GB) and zero-byte.
- Type: ordinary documents, images, media and archives; excluded temporary files.
- Network: good and slow; no class for intermittent or interrupted connections.
- The pattern shows that developers created classes for typical cases only and did not define classes for extreme size, special file names and unreliable networks.

iii) Reconstructed testing strategy

- **ECP with additional classes:** zero-byte, small, large, very large; ordinary, special-name, long-name, locked, hidden and temporary files; fast, slow, intermittent, offline networks.
- **BVA on size:** 0 bytes, 1 byte, 100 MB − 1 byte, 100 MB, 100 MB + 1 byte (chunk threshold), chunk size ± 1, 2 GB − 1, 2 GB, 2 GB + 1; maximum supported file size and maximum + 1.
- **BVA on name and path length:** 255 characters, 256 characters, and maximum path length ± 1.
- **BVA on network and timeout:** transfer completing at timeout − 1 second, timeout and timeout + 1 second.
- **Stress testing:** thousands of small files at once, several large files at the same time, many devices syncing to one account, and repeated edit-and-sync cycles for several hours.
- **Fault injection:** drop the connection in the middle of a chunk, pause and resume, switch network, reduce bandwidth to 256 kbps, fill the disk and stop the process during sync.
- **Conflict and recovery tests:** simultaneous edits from two devices, delete on one and edit on another, restore from version history.
- **Verification:** compare checksums of source and destination files for every test, and compare the file counts at both ends.

iv) Justification

The reconstructed tests target exactly the thresholds that the defect pattern suggests (chunk size, 2 GB limit, timeout) and the missing classes. Stress and fault-injection tests reproduce real conditions in which files were lost. Checksum verification, retry of failed chunks, version retention and an explicit report of skipped files are the controls that the tests confirm, and together they make silent data loss detectable and prevent it in future releases.

10. A sequence of SQL queries generated by an e-commerce Search and Filter module is given without source code. Reverse engineer the search logic, identify the AND/OR/NOT decision logic, reconstruct a Decision Table based strategy for all search combinations, and evaluate completeness.

Answer:

The queries and their result sets show how the selections made by the user are translated into conditions. The logic below is inferred from a typical set of observed queries.

i) Inferred search logic

- Different filter groups are combined with AND: category = ‘Phones’ AND price BETWEEN 10,000 AND 30,000 AND rating ≥ 4.

- Multiple values within one group are combined with OR: brand IN (‘A’, ‘B’) behaves as brand = A OR brand = B.
- The “in stock only” option is a NOT condition: NOT (stock = 0), or stock > 0.
- An exclusion option such as “exclude refurbished” is NOT (condition = ‘refurbished’).
- A keyword is matched against name OR description with a LIKE condition, and the whole keyword condition is combined with the filters by AND.
- Sorting and paging are applied after filtering (ORDER BY and LIMIT), and an empty filter group means that no condition is added for that group.

ii) Conditions and actions for the decision table

- C1: keyword entered (Y/N).
- C2: category selected (Y/N).
- C3: price range given (Y/N).
- C4: one or more brands selected (Y/N).
- C5: in-stock only selected (Y/N).
- Action: build the query with the conditions that are Y (joined by AND, brands joined by OR) and return the matching products, or a “no results” message.

iii) Reconstructed decision table strategy

With five conditions the full table has $2^5 = 32$ rules. Each rule is a combination of Y and N values, for example: Rule 1 (N, N, N, N, N): no filter, all products are returned; Rule 2 (Y, N, N, N, N): keyword only; Rule 17 (N, Y, N, N, N): category only; Rule 32 (Y, Y, Y, Y, Y): all filters, the query contains every condition. The other rules are generated in the same way, and each has a test that checks the generated SQL and the returned product set against the expected set.

- Rules 1 to 32 are all executed at least once, so every combination is covered.
- Rules that give the same logical result because a filter is absent can be reduced to one representative test for a quick regression run (about 10 rules), but the full set is run before release.
- Extra tests per condition: keyword with special characters, quotation marks and very long text (the text must be treated as data); price range with minimum equal to maximum, minimum above maximum (error or empty result), zero and negative; one brand versus all brands selected; in-stock only with all products out of stock.
- Logic tests for NOT and OR: selecting brands A and B returns the union, not the intersection; the exclusion of refurbished items removes only those items.
- Result checks: count, correctness of each item against the conditions, sorting order, paging at page boundaries (last page, empty page).

iv) Evaluation of completeness

- All 32 combinations of the five filters are covered, so no combination of filter groups is missing.
- The model does not cover values inside a filter (for example many brands, many categories) or combinations with sorting and paging, so these are covered by ECP, BVA and a few additional tests above.
- The model is based on inferred logic from the queries; if later queries show other behaviour (for example brand and category combined with OR), new rules must be added, so the table should be reviewed with the actual requirement or developer.
- Performance of the complex queries with a large data set is not evaluated by the decision table and requires separate load tests.

In conclusion, the decision table gives a complete, traceable set of tests for the filter combinations, and together with boundary tests on the price range and negative tests on the input, it provides a reliable verification of the search and filter module.

RUBRICS FOR CALCULATION:

Understanding of System (20) – Level 4 (Exemplary): Demonstrates clear and in-depth understanding of the system/product and its functionality; Level 3 (Proficient): Shows good understanding with minor gaps; Level 2 (Developing): Partial understanding; misses key aspects; Level 1 (Beginning): Lacks understanding of the system/product.

Decomposition (25) – Level 4 (Exemplary): Effectively breaks down the system into components with detailed analysis; Level 3 (Proficient): Decomposes system with minor gaps in analysis; Level 2 (Developing): Limited or incomplete decomposition; Level 1 (Beginning): Unable to analyze or break down system.

Identification of Components (20) – Level 4 (Exemplary): Accurately identifies all components and explains their functions clearly; Level 3 (Proficient): Identifies most components with minor errors; Level 2 (Developing): Identifies few components; explanations are weak; Level 1 (Beginning): Unable to identify components or functions.

Reconstruction (20) – Level 4 (Exemplary): Successfully reconstructs or models the system with accuracy and logic; Level 3 (Proficient): Reconstruction is mostly correct with minor issues; Level 2 (Developing): Partial reconstruction; lacks clarity; Level 1 (Beginning): Unable to reconstruct or model system.

Documentation (15) – Level 4 (Exemplary): Well-organized, clear documentation with diagrams and structured explanation; Level 3 (Proficient): Documentation is adequate with minor issues; Level 2 (Developing): Poorly organized or incomplete documentation; Level 1 (Beginning): No proper documentation or presentation.